G H raisoni COLLEGE
Engineering and Management
Pune

G H RAISONI COLLEGE OF ENGINEERING AND MANAGEMENT, PUNE
(Empowered Autonomous Institute, Affiliated to SPPU , Pune)

Department of CSE[Data Science]

TAE-2

Submitted By: Krish Saoji
Roll No : P65
Subject = DBMS

Dairy Booth & Milk Delivery Management DB

Domain & problem

- Local dairy booth serving ~40-50 households; deliveries tracked in a paper register
- Disputes over delivered quantity; monthly bill takes an evening of manual addition
- Goal: replace the register with a normalised relational DB (3NF/BCNF)

Key business rules (TAE 1)

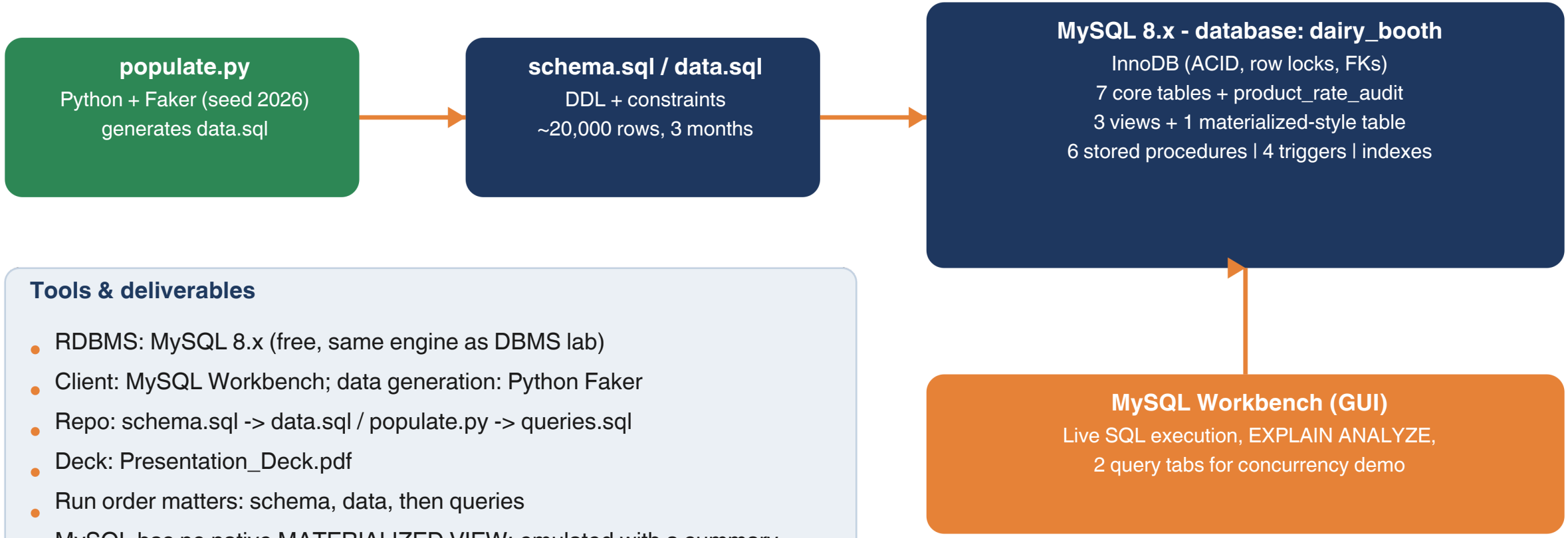
- Customer belongs to exactly one area; agent serves an area
- Customer - Product is M:N, resolved by SUBSCRIPTION
- Delivery only against an ACTIVE subscription
- Amount stored at delivery time (rates change)
- Delivery logs are never deleted (billing history)

Final 3NF relational schema (7 tables)

AREA	Area_ID, Area_Name, Pincode
CUSTOMER	Customer_ID, Cust_Name, Phone, Address, Area_ID*
DELIVERY_AGENT	Agent_ID, Agent_Name, Phone, Area_ID*
PRODUCT	Product_ID, Product_Name, Unit, Rate_Per_Unit
SUBSCRIPTION	Subscription_ID, Customer_ID*, Product_ID*, Daily_Qty, Start_Date, Status
DELIVERY_LOG	Log_ID, Subscription_ID*, Agent_ID*, Delivery_Date, Qty_Delivered, Amount, Delivery_Status
PAYMENT	Payment_ID, Customer_ID*, Bill_Month, Total_Amount, Amount_Paid, Payment_Date, Payment_Status

* = foreign key | + product_rate_audit (audit table filled by trigger)

System Architecture & Implementation Setup



Tools & deliverables

- RDBMS: MySQL 8.x (free, same engine as DBMS lab)
- Client: MySQL Workbench; data generation: Python Faker
- Repo: schema.sql -> data.sql / populate.py -> queries.sql
- Deck: Presentation_Deck.pdf
- Run order matters: schema, data, then queries
- MySQL has no native MATERIALIZED VIEW: emulated with a summary table + refresh procedure

Live DBMS Demonstration Plan

0:00	**DDL**	Run schema.sql: 7 tables, PK/FK (ON DELETE RESTRICT, ON UPDATE CASCADE), NOT NULL, UNIQUE, CHECK, DEFAULT, ALTER TABLE rules
1:00	**DML**	Load data.sql, show row counts: 150 customers, 303 subscriptions, 18,995 delivery logs, 398 payments
2:00	**Adv. SQL**	Joins (inner, outer, self), correlated subqueries, GROUP BY / HAVING, 3 views + materialized-style table
3:00	**Proc + Trigger**	Sec. 7: register customer, sp_record_delivery x3 -> TRG2 auto-creates bill -> sp_record_payment; rate revision -> audit trigger
4:15	**Rules**	Trigger blocks delivery on paused sub and log deletion; FK RESTRICT blocks customer delete
5:00	**EXPLAIN**	EXPLAIN ANALYZE before/after CREATE INDEX on 2-3 queries (full scan -> index range scan)

Advanced SQL Implemented

Joins

- 6-table INNER (delivery register)
- Substitute-agent join (2 agent aliases)
- LEFT OUTER: areas with 0 customers
- Anti-join: customers with no June bill
- FULL OUTER emulation; 2 SELF joins

Subqueries & Aggregates

- Bill above OWN area average (correlated)
- Max daily qty per product (correlated)
- EXISTS: paused subscription + dues
- GROUP BY / HAVING: area revenue, skip %
- Collection efficiency per month

Procedures (6)

- sp_record_delivery (txn + rollback)
- sp_generate_monthly_bill (idempotent)
- sp_record_payment (FOR UPDATE lock)
- sp_revise_product_rate
- sp_reassign_agent, sp_refresh_mv

Triggers (4)

- BEFORE INSERT: only ACTIVE subscription
- AFTER INSERT: running monthly bill
- AFTER UPDATE: product rate audit
- BEFORE DELETE: logs never deleted

Views (3 + 1)

- vw_customer_bill_status
- vw_agent_performance
- vw_product_monthly_revenue
- mv_area_monthly_revenue (refreshed)

Performance Tuning with Indexes

Query	Index created	BEFORE	AFTER
Q1 skipped deliveries / day (status = const, date range)	idx_log_status_date (delivery_status, delivery_date)	type ALL, full scan ~18,998 rows read, filesort	type range, covering index 181 rows read
Q3 one agent's deliveries in a date window	idx_log_agent_date (agent_id, delivery_date)	ref on FK index only 1,171 rows read	range on composite index 93 rows read
Q2 unpaid bills of a month (join payment/customer/area)	idx_pay_month_status (bill_month, payment_status)	scan customer (151 rows), probe payment by customer_id	ref on new index 67 rows, no customer scan

Why these indexes?

- Composite column order: equality column FIRST, range column SECOND (status, then date).
- FK columns get an automatic single-column index; a composite (agent_id, date) also serves range filters.
- Q2 shows a smaller gain: payment has only ~400 rows. Indexes matter most on large tables.

Numbers from a test run on the generated dataset; re-run on your MySQL and quote your own EXPLAIN ANALYZE output.

Viva Preparation

Slide 6 | Indexingchoices,ACIDcompliance, concurrency control (2 min)

Indexing choices

- B-tree indexes on filter columns, not on every column
- Composite: equality first, range second (leftmost-prefix rule)
- UNIQUE(subscription_id, delivery_date) is also an index (one log/day)
- Covering index avoids table lookups (Using index)
- Trade-off: extra writes and storage on 19k-row log

ACID compliance

- A: sp_record_delivery / sp_record_payment use START TRANSACTION + EXIT HANDLER -> ROLLBACK
- C: PK/FK/CHECK/UNIQUE + triggers keep data valid
- I: InnoDB, REPEATABLE READ default (MVCC)
- D: redo log + doublewrite; committed data survives crash
- Demo: Sec. 7.7, rolled-back UPDATE leaves no trace

Concurrency control

- SELECT ... FOR UPDATE locks the bill row inside sp_record_payment
- Two sessions paying same bill: second WAITS, no lost update
- Row-level locks in InnoDB, not table locks
- Over-payment blocked: check runs after the lock
- Demo: Sec. 7.8, two Workbench tabs